Universidad del Valle de Guatemala

Facultad de Ingeniería

Reporte Escrito

MLflow en la plataforma Databricks

Machine Learning Engineering

Integrantes:

Andy Fuentes — 22944
Diederich Solís — 22952
Diego Linares — 221256
Christian Echeverria

Fecha: octubre de 2026

1 Introducción y alcance

Este informe investiga el uso de MLflow en Databricks a partir del pipeline de Champions League del Taller 3 y la Actividad 9, publicado en el repositorio original de Diego Linares [6]. Se analiza su relación con CRISP-DM y la integración del pipeline mediante un notebook y un archivo Python.

MLflow permite organizar experimentos y empaquetar modelos; Databricks integra estas capacidades con su plataforma de datos. El registro gobernado en Unity Catalog y la publicación mediante Model Serving permiten continuar el ciclo de vida después del entrenamiento [1, 3].

1.1 £Qué usos puede tener para proyectos de Machine Learning?

- **Comparación de experimentos:** conservar hiperparámetros y resultados para elegir una configuración con evidencia [2].
- **Trazabilidad:** relacionar cada ejecución con métricas, archivos y contexto del entrenamiento [2].
- **Reutilización:** guardar y cargar un pipeline completo, incluyendo transformaciones y clasificador [5].
- **Colaboración:** centralizar versiones y controlar su acceso mediante Unity Catalog [3].
- **Operación:** integrar modelos con servicios de inferencia; publicar un modelo requiere configurar el servicio adicional [1].

1.2 Implementación de referencia: Taller 3

La base es el trabajo *MLflow con scikit-learn* proporcionado por el equipo [6]. Utiliza el dataset de partidos de Champions League y compara SVC, regresión logística, HistGradientBoosting y Random Forest. El repositorio ya contiene un notebook de integración, módulos Python, seguimiento de datos y registro de modelos.

Por ello, el objetivo de esta entrega es trasladar esa implementación a Databricks: conservar el pipeline y cambiar la configuración de seguimiento, las rutas del experimento y, si se utiliza, el registro gobernado en Unity Catalog. La integración reutiliza el preprocesamiento, los clasificadores y las funciones de seguimiento desarrolladas en el taller.

1.3 Usos en este proyecto

Se pueden comparar familias por F1 macro, identificar los datos usados por cada run y recuperar el modelo seleccionado por versión o alias. La misma base permite documentar cambios del preprocesamiento y repetir búsquedas de hiperparámetros. Las variables incluyen estadísticas observadas durante el partido. Por ello, esta evaluación clasifica resultados con datos del encuentro; no demuestra capacidad de pronóstico antes del partido.

2 £Qué etapas de CRISP-DM puede cubrir?

La siguiente relación es un análisis metodológico del equipo. MLflow apoya especialmente modelado y evaluación; junto con Databricks también participa en preparación y despliegue. El criterio del negocio y la interpretación de datos siguen siendo responsabilidad del equipo.

Etapa	Aplicación concreta	Cobertura
Comprensión del negocio	Documentar objetivo, supuestos y criterio de éxito en notas o etiquetas. La herramienta no define la necesidad del negocio.	Indirecta
Comprensión de los datos	Conservar referencias y archivos de diagnóstico. El análisis exploratorio se realiza con notebooks y herramientas de datos.	Parcial
Preparación de los datos	Guardar el preprocesamiento dentro del pipeline. Feature Store añade reutilización de variables en la plataforma.	Parcial
Modelado	Registrar cada configuración, entrenar modelos y conservar el pipeline y sus dependencias.	Directa
Evaluación	Comparar métricas de validación y conservar resultados del conjunto de prueba; revisar también criterios de negocio.	Directa
Despliegue	Versionar el modelo y cargarlo para inferencia. Model Serving y Jobs requieren configuración adicional en Databricks.	Con integración

2.1 Correspondencia con Champions League

1. **Preparación:** eliminar filas vacías y columnas que revelan el resultado; convertir porcentajes y razones; imputar, escalar y codificar categorías dentro del pipeline.
2. **Modelado:** ajustar las familias mediante RandomizedSearchCV, con F1 macro y validación estratificada.
3. **Evaluación:** comparar validación y prueba con la referencia de clase mayoritaria; conservar la matriz de confusión.
4. **Despliegue:** el taller registra la SVC con el alias `campeon`; en Databricks debe configurarse el registro correspondiente.

3 Adaptación del pipeline de Champions League

3.1 Estructura de la implementación

El notebook `03_pipeline_sklearn_mlflow.ipynb` integra el pipeline del taller. El archivo `seguimiento.py` contiene `entrenar_familia`, que registra parámetros, métricas, datasets, matriz de confusión, resultados de búsqueda y el modelo completo [6].

El pipeline usa `ColumnTransformer` para variables numéricas, porcentajes, razones y categorías; aplica imputación, escalado y One-Hot Encoding, y después `SelectKBest` y el clasificador. Conserva los transformadores propios del paquete.

3.2 Resultados reportados por el equipo

Familia	F1 macro CV	Exactitud prueba	F1 macro prueba
SVC	0.638	0.655	0.601
Regresión logística	0.611	0.621	0.542
HistGradientBoosting	0.600	0.621	0.579
Random Forest	0.583	0.621	0.576

Fuente: Taller 3 original, registro manual, página 10 [6]. Estas cifras pertenecen al trabajo original; no son resultados de esta adaptación ni de una ejecución en Databricks.

3.3 Cambio para ejecutar en Databricks

La integración utiliza el servidor de seguimiento administrado de Databricks y una ruta de experimento dentro del workspace:

```
from taller3_pipeline.seguimiento import (
    configurar_mlflow, entrenar_familia)

# La ruta del experimento se configura por usuario.
configurar_mlflow(
    experimento="/Users/usuario/Champions_Resultado",
    uri="databricks")
resultado = entrenar_familia(
    "svc", n_iter=40, folds=5, n_jobs=1)
```

El notebook de entrega importa la función del archivo Python de la raíz; ambos reutilizan el mismo paquete instalado y su CSV incluido. La lectura del dataset no depende del directorio de trabajo. La URI de seguimiento es configurable para validar localmente con SQLite y ejecutar después con el servicio administrado de Databricks. El registro opcional en Unity Catalog requiere un nombre `catalogo.esquema.modelo` y permisos [3]. El alcance de la implementación comprende Experiments, Tracking, Dataset Tracking y MLflow Models. Se guarda el pipeline con su firma, ejemplo de entrada y código de los transformadores, y se comprueba su recarga calculando nuevamente el F1 macro de prueba. La implementación se entrega preparada para Databricks; no se ha ejecutado ni validado en ese servicio. El registro de una versión en Unity Catalog es opcional y no equivale a desplegar un endpoint.

4 Conclusiones

MLflow permite conservar la relación entre datos, hiperparámetros, métricas y modelos. En el proyecto de Champions League facilita comparar familias con F1 macro y recuperar el pipeline completo, incluyendo sus transformaciones.

La integración con Databricks centraliza los experimentos en un workspace y utiliza el mismo código de entrenamiento desde un notebook o un archivo Python. El registro manual permite conservar la métrica de validación, los resultados de prueba y los artefactos de evaluación de forma explícita.

MLflow ofrece apoyo directo en modelado y evaluación dentro de CRISP-DM. Las herramientas de datos y gobierno de Databricks amplían su participación en preparación y despliegue. El valor de la solución consiste en mantener un proceso trazable y reproducible, con una selección de modelos basada en evidencia.

5 Repositorio y código del proyecto

Repositorio de esta entrega:
https://github.com/Andyfer004/Ejercicio5-MLOPS
Notebook de entrega:
notebooks/Champions_Databricks.ipynb
Archivo Python:
pipeline_champions_databricks.py
Paquete y notebook de referencia:
`src/taller3_pipeline/` y `notebooks/03_pipeline_sklearn_mlflow.ipynb`.
Notebook en Databricks: el enlace se incorporará después de ejecutar ambos formatos y compartir el notebook con el docente. No hay un enlace ni resultados de Databricks disponibles en esta preparación.
Procedencia y créditos: pipeline y dataset reutilizados del Taller 3, publicado por Diego Linares, con autoría de Diego Linares, Andy Fuentes, Diederich Solís y Christian Echeverria. Versión de referencia: `bd619c3cea3e`. El README detalla la instalación y ejecución de ambos formatos.

6 Referencias oficiales

[1] Databricks. *MLflow on Databricks*.
https://docs.databricks.com/aws/en/mlflow/

[2] Databricks. *Track model development using MLflow*.
https://docs.databricks.com/aws/en/mlflow/tracking

[3] Databricks. *Manage model lifecycle in Unity Catalog*.
https://docs.databricks.com/aws/en/machine-learning/manage-model-lifecycle/

[4] Databricks. *Databricks Feature Store*.
https://docs.databricks.com/aws/en/machine-learning/feature-store/

[5] MLflow. *mlflow.sklearn: Python API*.
https://mlflow.org/docs/latest/api_reference/python_api/mlflow.sklearn.html

[6] Equipo del curso. *Taller 3: MLflow con scikit-learn*. Octubre de 2026. PDF original conservado en referencias/Taller3_MLOPS.pdf y repositorio original: https://github.com/DiegoLinares11/MLFlow

Consulta: 16 de octubre de 2026.